运动模糊点扩散函数的离散实现

姓名：蔡斌　班级：数学 jd2401

学号：1024001338

2026 年 11 月 2 日

1　运动模型

课上推导了匀速直线运动的连续核。本次考虑运动方向，并比较离散核与 MATLAB 的 fspecial ('motion',LEN,THETA)。LEN 表示运动长度，单位为像素；THETA 从直角坐标系的水平正方向起算，逆时针为正。因此，0° 是水平运动，90° 是竖直运动。

把运动中点放在原点，单位方向向量为 $(\cos\theta, \sin\theta)$。连续线段为

$$\mathbf{r}(s) = (s\cos\theta, s\sin\theta), \qquad -\frac{L}{2} \le s \le \frac{L}{2}. \tag{1}$$

理想 PSF 沿这条线段均匀分布：

$$h(x,y) = \frac{1}{L}\int_{-L/2}^{L/2} \delta(x - s\cos\theta)\,\delta(y - s\sin\theta)\,\mathrm{d}s. \tag{2}$$

连续模型确定了运动轨迹，但没有规定像素网格上的边界采样和端点权重。这些离散细节会改变最终矩阵。

2　离散核的构造

对一个像素中心 (x, y)，沿运动方向的坐标为 $q = x\cos\theta + y\sin\theta$，到运动直线的垂直距离为 $d_\perp = |-x\sin\theta + y\cos\theta|$。到有限线段的距离为

$$d = \sqrt{d_\perp^2 + \max\left(|q| - \frac{L}{2}, 0\right)^2}. \tag{3}$$

可以按距离给像素分配权重，再归一化。但 MATLAB 还要确定核尺寸、端点处理和镜像顺序。R2023b 的实现先生成最小半核，再计算距离；端点附近改用欧氏距离，随后转换成一像素宽的线性权重、镜像并归一化。代码保留了其中的 `eps` 修正项。[1]

[1] MathWorks, *fspecial: Create predefined 2-D filter*, `https://www.mathworks.com/help/images/ref/fspecial.html`.

3 MATLAB 对照

AI 初版按到有限线段的距离给方阵赋权。对 LEN=9、THETA=45°，初版支持域为 11×11，MATLAB 输出为 7×7。差异来自核尺寸的确定方式：MATLAB 根据角度生成最小半核网格。

最终实现依次计算半核边界、端点欧氏距离和像素权重，再镜像展开并归一化。水平、竖直运动分别输出行向量和列向量。

AI 交互

交互从题目中的“直接编写运动 PSF”开始。AI 给出距离采样初版，MATLAB 对照显示 45° 核尺寸不同；随后围绕半核网格和端点处理改写函数，再用多组 LEN、THETA 复测。

对话截图

截图 1：原始提问与 AI 初版

截图 2：尺寸差异与修改分析

截图 3：最终函数与 MATLAB 测试结果

4 函数实现

```
function h = motion_psf(len, theta)
%MOTION_PSF Motion-blur PSF compatible with MATLAB R2023b fspecial.
% LEN is in pixels; THETA is counter-clockwise from the horizontal axis.
narginchk(0, 2);
if nargin < 1, len = 9; end
if nargin < 2, theta = 0; end
validateattributes(len, {'double'}, ...
    {'positive','finite','real','nonempty','scalar'}, mfilename, 'LEN', 1);
validateattributes(theta, {'double'}, ...
    {'finite','real','nonempty','scalar'}, mfilename, 'THETA', 2);
len = max(1, len);
halfLength = (len - 1) / 2;
phi = mod(theta, 180) / 180 * pi;
cosPhi = cos(phi);
sinPhi = sin(phi);
xSign = sign(cosPhi);
lineWidth = 1;

% Smallest half-grid for this angle.
sx = fix(halfLength*cosPhi + lineWidth*xSign - len*eps);
sy = fix(halfLength*sinPhi + lineWidth - len*eps);
[x, y] = meshgrid(0:xSign:sx, 0:sy);

% Perpendicular distance, with endpoint correction.
distToLine = y*cosPhi - x*sinPhi;
radialDistance = sqrt(x.^2 + y.^2);
lastPixel = find((radialDistance >= halfLength) & ...
    (abs(distToLine) <= lineWidth));
parallelEndpointDistance = halfLength - abs( ...
    (x(lastPixel) + distToLine(lastPixel)*sinPhi) / cosPhi);
distToLine(lastPixel) = sqrt( ...
    distToLine(lastPixel).^2 + parallelEndpointDistance.^2);

% Convert distance to weights, mirror, and normalize.
distToLine = lineWidth + eps - abs(distToLine);
distToLine(distToLine < 0) = 0;
h = rot90(distToLine, 2);
h(end+(1:end)-1, end+(1:end)-1) = distToLine;
h = h ./ (sum(h(:)) + eps*len*len);
if cosPhi > 0, h = flipud(h); end
end
```

5 结果验证

测试程序将函数与本机 MATLAB R2023b 的 `fspecial` 比较。每组先检查矩阵尺寸，再用 `isequal` 检查元素。测试参数为：

- LEN：$[0.25, 1, 1.25, 2.5, 5, 9, 13.7, 20]$；
- THETA：$[-180, -90, -45, 0, 30, 45, 89.5, 90, 135, 180, 225, 270, 360]$ 度。

共比较 $8 \times 13 = 104$ 组，另检查无参数调用和省略 THETA 的默认调用。

检查内容	通过数量	最大绝对误差
LEN/THETA 参数组合	104/104	0
默认参数调用	2/2	0

LEN=9 时几个方向的输出如下。系数和略小于 1，是 MATLAB 归一化时加入浮点修正项所致；本函数与内置输出逐元素相同。

THETA	矩阵大小	系数和
0°	1×9	0.99999999999999822
45°	7×7	0.99999999999999833
90°	9×1	0.99999999999999833

图中把核居中放入 9×9 矩阵显示；测试比较的是未补零的原始输出。

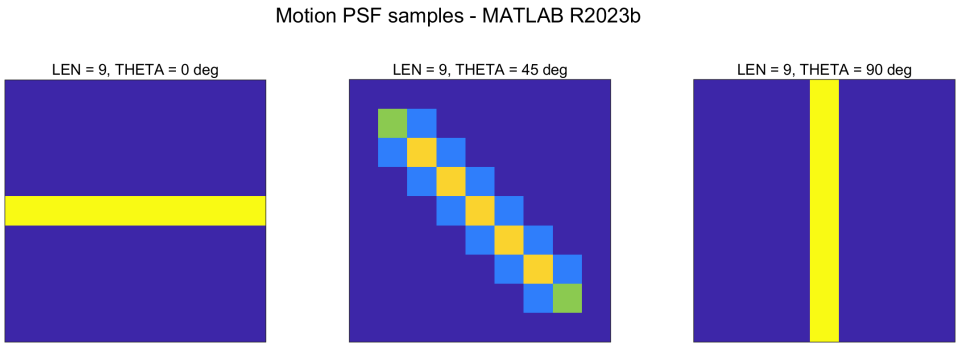


图 1: LEN=9 时 THETA 为 0°、45° 和 90° 的离散核

验证环境：MATLAB R2023b（23.2.0.2365128）。

测试脚本：`test_motion_psf.m`。